

INTRODUCCIÓN AL ANÁLISIS DE DATOS Y HERRAMIENTAS BÁSICAS

ANÁLISIS DE DATOS – EXPLORADOR – MISIÓN 1

¿Qué es el análisis de datos?

El análisis de datos es el proceso de examinar, limpiar, transformar y modelar datos con el objetivo de descubrir información útil, sacar conclusiones y apoyar la toma de decisiones.



Recolección de datos (obtener datos de encuestas, sensores, bases de datos)



Limpieza de datos (corregir errores, eliminar duplicados)



Exploración de datos (buscar patrones, calcular promedios, etc.)



Visualización de datos (gráficos que resumen los datos)



Toma de decisiones (usar lo aprendido para actuar)

¿Por qué es importante el análisis de datos?

El análisis de datos convierte la información en conocimiento útil para tomar mejores decisiones.

Negocios: *¿Por qué una tienda vende más un producto?*

Salud: *¿Qué tratamientos son más efectivos según los datos?*

Deportes: *¿Qué jugadas aumentan la probabilidad de ganar?*

Evita decisiones basadas en suposiciones

- Apoya con hechos y evidencia.

Identifica patrones y oportunidades

- Encuentra lo que funciona y lo que no.

Detecta errores o riesgos a tiempo

- Prevenir es mejor que corregir tarde.

Optimiza recursos y procesos

- Ahorra tiempo, dinero y esfuerzo.

Aplicaciones del análisis de datos

El análisis de datos se usa en casi todas las industrias para resolver problemas, optimizar procesos y tomar decisiones inteligentes.

Marketing: ¿Qué tipo de cliente compra más?

- Segmentación, campañas personalizadas.

Salud: ¿Qué tratamiento es más eficaz?

- Diagnóstico basado en datos médicos.

Deportes: ¿Qué jugador rinde mejor?

- Estrategias y rendimiento.

Negocios: ¿Qué producto vende más y cuándo?

- Predicción de demanda, gestión de inventarios.

Educación: ¿Qué estudiantes están en riesgo de abandonar?

- Seguimiento de desempeño.

Transporte: ¿Cómo mejorar rutas?

- Optimización logística y reducción de tiempos.

A close-up photograph of a snake's head, showing detailed scales in shades of yellow, orange, and brown. The snake's eye is visible, and the background is dark.

INTRODUCCIÓN PYTHON

¿Qué es Python?

Python es un lenguaje de programación interpretado, de alto nivel, diseñado para ser fácil de leer y escribir..

Sintaxis simple: más parecido al inglés que otros lenguajes.

Multiplataforma: funciona en Windows, macOS y Linux.

Interpretado: no necesita compilarse para ejecutarse.

Multipropósito: se usa en web, ciencia de datos, automatización, videojuegos, inteligencia artificial y más

¿Por qué usar Python en análisis de datos?



Popularidad en la industria

Python es el lenguaje más utilizado en análisis de datos y ciencia de datos, según encuestas como Stack Overflow y Kaggle.



Facilidad de aprendizaje

Su sintaxis es clara y directa, ideal para principiantes, la cual permite enfocarse en resolver problemas, no en memorizar reglas complicadas.



Librerías especializadas para análisis de datos

Python tiene herramientas potentes y gratuitas para cada etapa del análisis: ***pandas***, ***numpy***, ***matplotlib***, ***seaborn***, ***scikit-learn***.



Comunidad activa y recursos abundantes

Hay miles de tutoriales, foros, cursos gratuitos y notebooks compartidos. Esto facilita resolver problemas, aprender rápido y colaborar.

¿Qué se puede hacer con Python en análisis de datos?



Explorar y entender datos

- Ver las primeras filas de una tabla, calcular promedios, encontrar valores nulos o duplicados.
- **Ejemplo:** *¿Cuál es la edad promedio de los clientes? ¿Qué productos tienen más ventas?*

Visualizar datos

- Crear gráficos que ayuden a entender patrones, comparaciones o tendencias.
- **Ejemplo:** *Graficar la evolución de ventas en el tiempo o comparar regiones.*

Limpiar y transformar datos

- Eliminar errores, cambiar formatos, crear nuevas columnas o agrupar información.
- **Ejemplo:** *Corregir nombres mal escritos o convertir fechas en día/mes/año.*

Modelar y predecir

- Aplicar algoritmos para hacer predicciones, clasificar o segmentar datos.
- **Ejemplo:** *Predcir si un cliente dejará un servicio, o agrupar tipos de usuarios.*

Automatizar tareas repetitivas

- Generar reportes, enviar emails, convertir archivos, etc.
- **Ejemplo:** *Crear un informe mensual automáticamente y enviarlo por correo.*

Buenas prácticas iniciales

- Usa nombres **claros** y **descriptivos** para tus variables.
- Comenta partes importantes del código.
- Usa **sangrías (indentación)** correctamente (*muy importante en Python*).
- **Evita copiar y pegar sin entender.**
- Ejecuta tu código por partes y **lee bien los errores** (*¡todos los cometemos!*).

Inicio de Python

¿QUÉ ES UN COMENTARIO?

- Es una línea de texto que no se ejecuta.
- Se usa para explicar el código, documentar y dejar notas para ti o para otras personas.
- Los comentarios mejoran la legibilidad del código.
- Los comentarios comienzan con “#” y Python los ignorará

```
# Esto es un comentario
```

¿QUÉ ES `PRINT()`?

- Es una función incorporada que **muestra información en pantalla.**
- Ideal para ver resultados o mensajes durante la ejecución del código.

```
print("Hola mundo")           # Texto simple
print("Suma:", 3 + 4)         # Múltiples elementos
print("Resultado:", resultado) # Mostrar una variable
```

Entrada de datos – `input()`

- Es una función que **pide información al usuario** desde el teclado.
- Siempre **retorna un string (texto)**, aunque parezca un número.

```
nombre = input("¿Cómo te llamas? ")  
print("Hola", nombre)
```

```
edad = input("Edad: ")           # esto es texto  
edad = int(edad)                 # ahora es número entero  
  
# o directamente  
edad = int(input("Edad: "))
```

Tipos de datos en Python

Tipo	Descripción	Ejemplo
int	Entero (número sin decimales)	5, -3, 100
float	Número decimal	3.14, -0.5
str	Texto (cadena de caracteres)	'Hola', "123"
bool	Valor lógico	True, False

```
type(edad)          # Muestra <class 'int'>
```

Variables en Python

¿Qué es una variable?

Es un espacio en memoria donde se guarda información para usarla después.

Reglas de nombres:

- ✓ No debe comenzar con número.
- ✓ No puede tener espacios ni símbolos especiales (excepto _).
- ✓ Debe ser descriptiva.

```
nombre = "Carlos"  
edad = 31  
altura = 1.75
```

Concatenación

¿Qué es concatenar?

Unir valores en una sola salida o string.

```
nombre = "Luis"
print("Hola " + nombre)    # Hola Luis

edad = 25
print("Tienes", edad, "años")  # Más flexible

print("Edad: " + str(edad))    # str() convierte número a texto
```

Actividad básica inicio con python

1. Pídele al usuario que escriba su nombre y su ciudad. Luego muestra un saludo como:
 - *"Hola Ana, saludos desde Bogotá."*
2. Solicita el año actual y el año de nacimiento del usuario. Calcula y muestra su edad.
 - *"Tienes 25 años."*
3. Pide al usuario su estatura en metros y muestra su estatura en centímetros.
 - *"Tu estatura es 175.0 cm"*
4. Solicita al usuario su nombre, edad y carrera. Luego muestra un resumen como:
 - *"María tiene 22 años y estudia Ingeniería."*

Listas en Python

¿Qué es una lista?

- Una lista es una colección **ordenada y modificable** de elementos.
- Puede contener distintos tipos de datos (números, texto, booleanos, incluso otras listas).

```
mi_lista = [10, 20, 30, 40]

nombres = ["Ana", "Luis", "Sofía"]

mixta = [1, "texto", True]

lista_lista = ["Carlos", [1, 11, 111], "Maria"]
```

Acceso a elementos de una lista

- Se accede a los valores por medio de los índices los cuales pueden ser números positivos o negativos.

```
nombres = ["Ana", "Luis", "Sofía"]

print(nombres[0]) # Ana
print(nombres[2]) # Sofía
print(nombres[-1]) # Último elemento: "Sofía"
```

Modificación, agregación y eliminación de datos de una lista

Las listas son mutables, lo que significa que pueden cambiar después de crearse.

```
nombres[1] = "Carlos"
print(nombres) # ['Ana', 'Carlos', 'Sofía']
```

Agregar datos a una lista

- Usar **append()** para agregar al final

```
nombres.append("Lucía")
print(nombres) # ['Ana', 'Carlos', 'Sofía', 'Lucía']
```

- Usar **insert()** para agregar en una posición específica

```
nombres.insert(1, "Pedro")
print(nombres) # ['Ana', 'Pedro', 'Carlos', 'Sofía', 'Lucía']
```

Eliminar datos de una lista

- Usar **remove()** para eliminar por valor, si el valor no existe, da error.

```
nombres.remove("Carlos")
print(nombres) # ['Ana', 'Pedro', 'Sofía', 'Lucía']
```

- Usar **pop()** para eliminar por índice, si no se especifica un índice, **pop()** elimina el último elemento.

```
nombres.pop(2) # Elimina a 'Sofía'
print(nombres) # ['Ana', 'Pedro', 'Lucía']
```

Actividad básica lista

1. Crea una lista con 5 productos que comprarías en un supermercado. Luego:
 - Muestra el primer y último producto.
 - Cambia el tercer producto por otro diferente.
 - Muestra la lista final.
2. Crea una lista con 3 nombres de estudiantes. Luego:
 - Agrega 2 nuevos estudiantes al final.
 - Inserta uno en la segunda posición.
 - Elimina al estudiante que está en la tercera posición.
 - Muestra la lista resultante.
3. Supón que registraste estas temperaturas (en °C) durante una semana
 - Muestra la temperatura del segundo y sexto día.
 - Elimina la primera aparición de la temperatura 25.
 - Agrega una nueva temperatura al final.
 - Muestra la lista modificada.

Diccionarios en Python

¿Qué es un diccionario?

- Es una estructura de datos que **almacena pares clave-valor**.
- Similar a una "tabla de referencia": se busca un valor usando una clave.
- Es útil cuando **necesitamos asociar un dato a un nombre único** (como el nombre de un estudiante con su nota, o un producto con su precio).

```
inventario = {  
    "productos" : ["Martillo", "Clavo", "Martillo"],  
    "precios" : [10000, 2000, 1500],  
    "cantidad" : [10, 15, 50]  
}
```

```
mi_diccionario = {  
    "nombre": "Ana",  
    "edad": 25,  
    "ciudad": "Bogotá"  
}
```

- Las claves suelen ser str, pero también pueden ser números o booleanos.
- **Los valores pueden ser de cualquier tipo:** número, texto, lista, otro diccionario, etc.

Acceder e imprimir un valor del diccionario

Ver todas las claves o valores

```
print(mi_diccionario.keys())    # dict_keys(['nombre', 'edad', ...])  
print(mi_diccionario.values())  # dict_values(['Ana', 26, ...])
```

Acceder a un valor por su clave, si la clave no existe, se produce un error (KeyError)

```
print(mi_diccionario["nombre"]) # Ana
```

Alternativa segura

```
print(mi_diccionario.get("nombre", "No encontrado"))
```

Agregar, modificar, eliminar valores y elementos.

Agregar o modificar valores

```
mi_diccionario["profesión"] = "Ingeniera" # agregar nueva clave  
mi_diccionario["edad"] = 26               # modificar valor existente
```

Eliminar elementos

- Con del

```
del mi_diccionario["ciudad"]
```

- Con .pop()

```
mi_diccionario.pop("edad")
```

Actividad básica diccionario

1. Crea un diccionario llamado **calificaciones** con los siguientes datos:

- Agrega un nuevo estudiante "**Pedro**" con las calificaciones [84, 89, 87].
- Elimina el estudiante "**Luis**" del diccionario.
- Muestra el diccionario actualizado.

```
calificaciones = {  
    "Ana": [90, 85, 88],  
    "Luis": [78, 80, 79],  
    "Carla": [92, 95, 91]  
}
```

2. Crea un diccionario llamado **compras** donde las claves sean categorías y los valores listas de productos:

- Agrega una nueva categoría "**verduras**" con los productos ["zanahoria", "brócoli"].
- Elimina la categoría "**limpieza**".
- Muestra el contenido final del diccionario.

```
compras = {  
    "frutas": ["manzana", "banano"],  
    "lácteos": ["leche", "yogur"],  
    "limpieza": ["jabón", "detergente"]  
}
```

Ciclos Repetitivos en Python

¿Qué es un ciclo?

Un ciclo permite **repetir una instrucción o bloque de código múltiples veces automáticamente**, sin necesidad de escribirlo una y otra vez.

Hay dos ciclos principales en Python:

- ***for*** → cuando sabes cuántas veces quieres repetir algo
- ***while*** → cuando repites algo hasta que una condición deje de cumplirse

Bucle **for** en análisis de datos

¿Por qué es útil en análisis de datos?

El bucle **for** se usa frecuentemente para:

- **Recorrer listas** de datos y aplicar cálculos o filtros.
- **Explorar diccionarios** que representan registros.
- **Limpiar datos manualmente** antes de usar librerías como **pandas**.
- **Simular operaciones** como sumar, contar o buscar valores.

```
for elemento in colección:  
    # hacer algo con cada elemento
```

Ejemplos for aplicados en lista

Recorrer lista de edades

Útil para revisar cada valor o aplicar condiciones (más adelante, con **if**).

```
edades = [18, 25, 30, 22, 17]
for edad in edades:
    print("Edad encontrada:", edad)
```

Calcular promedio de una lista

Muy común al procesar calificaciones, precios, etc.

```
notas = [3.5, 4.0, 2.8, 3.9]
suma = 0

for nota in notas:
    suma += nota

promedio = suma / len(notas)
print("Promedio:", promedio)
```

Ejemplos for aplicados en diccionario

Recorrer diccionario con datos de una persona

Útil cuando se desea mostrar todos los datos de un registro.

```
persona = {
    "nombre": "Juan",
    "edad": 30,
    "ciudad": "Bogotá"
}

for clave, valor in persona.items():
    print(clave, "→", valor)
```

Diccionario con listas – notas por estudiante

Muy común para calcular estadísticas de múltiples personas o productos.

```
calificaciones = {
    "Ana": [4.5, 4.0, 5.0],
    "Luis": [3.8, 4.1, 4.0]
}

for estudiante, notas in calificaciones.items():
    promedio = sum(notas) / len(notas)
    print(estudiante, "→ promedio:", round(promedio, 2))
```

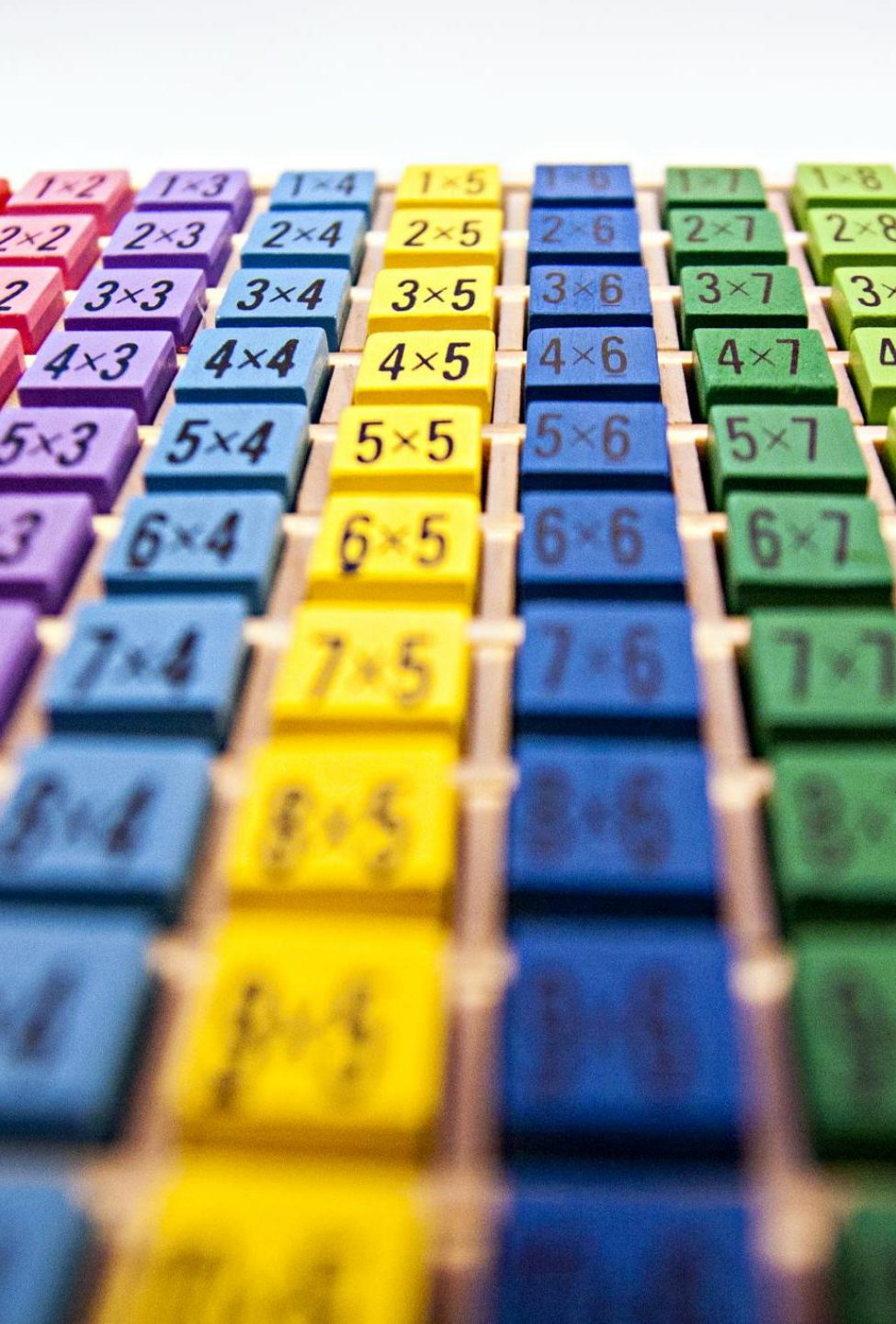
MANEJO DE LIBRERÍAS

Explicación de cada librería

Pandas es una de las librerías que Python que permite realizar análisis y manipulación de datos, es muy potente, flexible y fácil de usa.

NumPy al igual que pandas esta construida sobre Python, permite el manejo de matrices en n cantidad de dimensiones, además de facilitar el computo numero.





Introducción a numpy

¿Qué es numpy?

- numpy (*Numerical Python*) es una librería especializada en cálculos numéricos, que permite trabajar con arrays y operaciones matemáticas de forma rápida y eficiente.

¿Por qué usar numpy?

- Las listas de Python son flexibles pero lentas y no están optimizadas para operaciones numéricas.
- numpy es más rápido porque trabaja con arrays homogéneos y usa código optimizado en C.
- Permite operar con datos sin bucles, usando operaciones vectorizadas.

lista vs array

- La lista es una estructura de Python que puede contener diferentes tipos de datos.
- El array de *numpy* es una **estructura optimizada** para trabajar con números, y todos sus elementos son del mismo tipo.
- A primera vista parecen iguales, pero internamente el array permite operaciones más eficientes.

```
import numpy as np

lista = [1, 2, 3, 4, 5]
array = np.array([1, 2, 3, 4, 5])

print("Lista:", lista)
print("Array:", array)
```

Tipo de dato (type)

- La lista es una estructura nativa de Python.
- El array es del tipo *numpy.ndarray*, que es la **base de todas las operaciones matemáticas avanzadas** en numpy.
- Este tipo especial permite usar métodos como `.sum()`, `.mean()`, etc., directamente.

```
print(type(lista))    # <class 'list'>
print(type(array))    # <class 'numpy.ndarray'>
```

Operaciones de los elementos

- Con listas, necesitas un bucle manual (for) para sumar o multiplicar cada uno de los elementos.
- Con arrays, puedes usar `.sum()` directamente, que es más limpio, más rápido y más legible.
- Esta es una de las grandes ventajas de numpy: operaciones vectorizadas automáticas.

```
# Con lista
suma_lista = 0
for num in lista:
    suma_lista += num
print("Suma (lista):", suma_lista)

# Con array
print("Suma (array):", array.sum())
```

```
# Con lista
doble_lista = [x * 2 for x in lista]
print("Doble (lista):", doble_lista)

# Con array
doble_array = array * 2
print("Doble (array):", doble_array)
```

Atributos de un array

```
print("Forma:", array.shape)      # (5,)
print("Dimensiones:", array.ndim) # 1
print("Tipo de datos:", array.dtype) # int64 (puede variar)
print("Tamaño total:", array.size) # 5
```

.shape → muestra cuántos elementos tiene y cómo están organizados.

.ndim → dice cuántas dimensiones tiene el array (en este caso, 1).

.dtype → indica el tipo de dato contenido (por ejemplo, enteros).

.size → número total de elementos.

Indexación y Slicing

- Igual que con listas, puedes acceder a los elementos por su posición.
- Los índices negativos permiten contar desde el final.
- Acceder a datos individuales en un array es simple y eficiente.
- Puedes obtener porciones del array usando índices como con listas.
- El slicing es muy útil para seleccionar subconjuntos de datos sin tener que copiarlos manualmente.
- Se puede controlar el inicio, el final y el paso ([**inicio:fin:passo**]).

```
print(array[0])    # Primer elemento
print(array[-1])   # Último elemento
```

```
print(array[1:4])   # Del segundo al cuarto [2, 3, 4]
print(array[:3])    # Primeros tres [1, 2, 3]
print(array[::-2])  # Cada dos [1, 3, 5]
```

¿Qué es una matriz en numpy?

Es un array bidimensional: una tabla de números organizados en filas y columnas, como una hoja de cálculo.

- Cada sublista representa una fila.
- Este array tiene 2 filas y 3 columnas.

```
import numpy as np

matriz = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

print(matriz)
```

Atributos de una matriz

- .shape** muestra (filas, columnas).
- .ndim** confirma que es un array 2D.
- .size** indica cuántos elementos totales hay.

```
print("Forma:", matriz.shape)      # (2, 3)
print("Dimensiones:", matriz.ndim) # 2
print("Tamaño total:", matriz.size) # 6
print("Tipo de datos:", matriz.dtype)
```

Acceder, Slicing y Operaciones básicas con matrices

- Accedes usando **[fila, columna]**, mucho más limpio y directo que con listas anidadas ([][]).
- Puedes seleccionar **filas o columnas completas**.
- Muy útil para extraer subconjuntos sin copiar todo manualmente.
- Las operaciones pueden hacerse **por fila o por columna** con el argumento **axis**.

```
print(matriz[0, 1]) # Fila 0, columna 1 → 2
print(matriz[1, 2]) # Fila 1, columna 2 → 6
```

```
print(matriz[0])      # Primera fila completa: [1, 2, 3]
print(matriz[:, 1])   # Segunda columna completa: [2, 5]
print(matriz[1, :2])  # Fila 1, primeras dos columnas: [4, 5]
```

```
print("Suma total:", matriz.sum())
print("Suma por fila:", matriz.sum(axis=1))
print("Suma por columna:", matriz.sum(axis=0))
```

Actividad básica con numpy

Análisis de temperaturas (array)

Supón que registraste las temperaturas máximas de 7 días consecutivos:

```
import numpy as np
temperaturas = np.array([29, 31, 28, 30, 27, 32, 33])
```

Realiza las siguientes acciones:

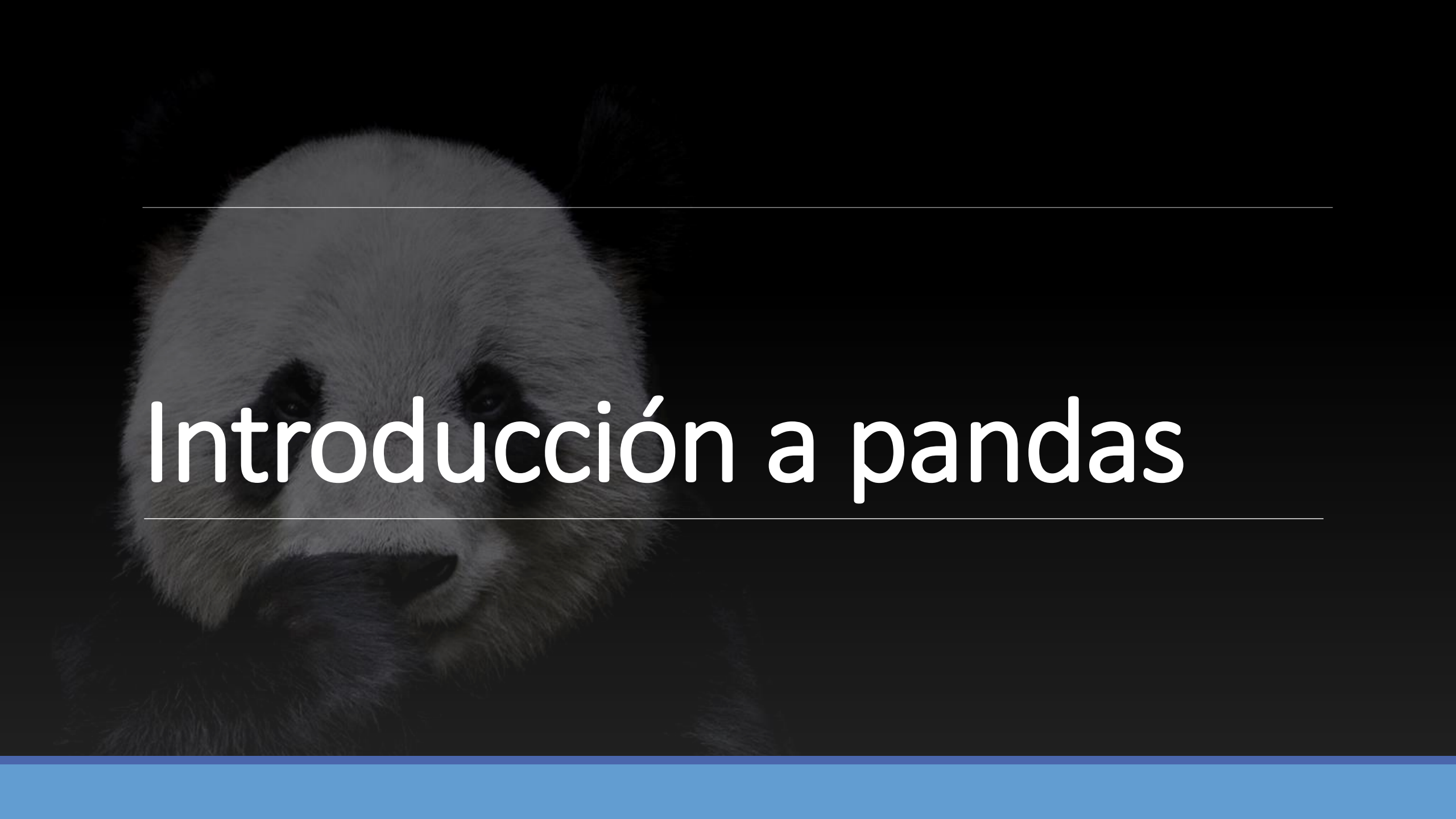
1. Muestra la temperatura del tercer día.
2. Calcula la temperatura promedio de la semana.
3. Muestra las temperaturas mayores a 30 grados.
4. Suma 1 grado a todas las temperaturas y muestra el nuevo array.

Análisis de ventas por región (matriz)

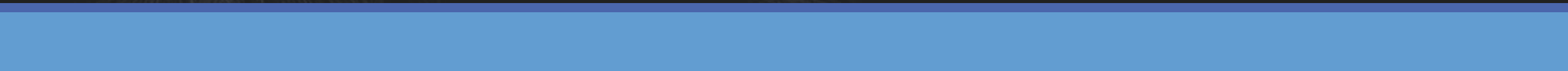
Tienes una matriz con las ventas (en miles de USD) de 3 regiones durante 4 trimestres:

```
ventas = np.array([
    [120, 135, 150, 160], # Región Norte
    [100, 110, 115, 130], # Región Sur
    [90, 105, 100, 120]   # Región Centro
])
```

1. Muestra las ventas del segundo trimestre para todas las regiones.
2. Calcula las ventas totales por región.
3. Calcula las ventas totales por trimestre.



Introducción a pandas



¿Qué es pandas?

pandas es una **librería de Python especializada en el análisis, manipulación y limpieza de datos** estructurados, como los que vienen en archivos .csv, Excel, bases de datos, etc.

Su nombre viene de "**panel data**", y es ampliamente usada en ciencia de datos, estadísticas, finanzas, salud, marketing, etc.

¿Por qué es importante en análisis de datos?

- ✓ Es la herramienta **más usada** en análisis de datos con Python.
- ✓ Permite trabajar con datos tabulares (como en Excel o Google Sheets) de **forma programática y reproducible**.
- ✓ Facilita tareas que antes eran tediosas: limpieza, filtrado, ordenamiento, agrupación, cálculo de promedios, conteos, etc.

¿Qué se puede hacer con pandas?

- ✓ **Leer archivos** y transformarlos en tablas.
- ✓ **Explorar datos** (ver las primeras filas, tipos, valores faltantes).
- ✓ **Filtrar y seleccionar columnas o filas.**
- ✓ **Ordenar, agrupar y resumir** información.
- ✓ **Limpiar datos** (quitar duplicados, tratar nulos, cambiar formatos).
- ✓ **Crear columnas nuevas**, combinar tablas, aplicar funciones.

Importación de pandas

- **pandas** no viene con Python por defecto, pero ya está instalado en Google Colab.
- Por convención, se importa como **pd** para usarlo más fácilmente.

```
import pandas as pd
```

Introducción a Series de Pandas

Una Series es una estructura de datos unidimensional de Pandas, similar a una lista o un arreglo de Numpy, pero con etiquetas (índices) para cada elemento.

Es ideal para representar una columna de datos, como una lista de edades, precios, nombres, etc.

```
import pandas as pd

edades = pd.Series([25, 30, 28, 22, 35])
print(edades)
```

Índices en Series de pandas

Agregar índices personalizados

```
edades = pd.Series([25, 30, 28], index=["Ana", "Luis", "María"])  
print(edades)
```

Los índices son los números a la izquierda (0, 1, 2, ...) son valores automáticos.

Puedes acceder a valores como con una lista

```
print(edades[1]) # 30
```

Ahora puedes acceder por nombre

```
print(edades["Luis"]) # 30
```

Esto es muy útil cuando tienes datos con nombres, categorías o identificadores únicos.

¿Qué es un DataFrame?

Un DataFrame es una **tabla de datos**, con **filas e índices**, y **columnas con nombre**, es la estructura principal de trabajo en pandas.

Crear un DataFrame desde un diccionario

Explicación:

- Cada **clave del diccionario** es una **columna**.
- Cada **valor (lista)** representa los datos de esa **columna**.
- El DataFrame organiza todo en una **tabla ordenada**.

```
datos = {  
    "Nombre": ["Ana", "Luis", "María"],  
    "Edad": [25, 30, 28],  
    "Ciudad": ["Bogotá", "Medellín", "Cali"]  
}  
  
df = pd.DataFrame(datos)  
print(df)
```

Dataset base: convertir un diccionario en DataFrame

1. Creación de DataFrame

```
import pandas as pd

datos = {
    "Nombre": ["Ana", "Luis", "María", "Carlos", "Lucía"],
    "Edad": [25, 30, 28, 22, 35],
    "Ciudad": ["Bogotá", "Medellín", "Cali", "Barranquilla", "Bogotá"]
}

df = pd.DataFrame(datos)
print(df)
```

2. Ver las primeras y últimas filas

```
print(df.head())      # Primeras 5 filas
print(df.tail(2))     # Últimas 2 filas
```

3. Ver dimensiones y estructura básica

```
print("Filas y columnas:", df.shape)
print("Número de registros:", len(df))
```

.shape devuelve una tupla (filas, columnas).

Len(df) da solo el número de filas.

4. Tipos de datos de cada columna

```
print(df.dtypes)
```

Saber si las columnas están en el formato correcto.

Explorar datos del DataSet

Ver resumen de estructura con `.info()`

Incluye:

```
df.info()
```

- Número total de entradas.
- Columnas, nombres y tipo de dato.
- Cantidad de valores no nulos.
- Memoria usada.

Muy útil para detectar **valores faltantes**, ver **tipos de datos** y **verificar columnas**.

Estadísticas rápidas con `.describe()`

Muestra:

```
df.describe()
```

- Promedio, desviación estándar, mínimo, percentiles y máximo de **columnas numéricas**.
- Ideal para tener una **visión general rápida de los datos numéricos**.

Modificación de columnas en el DataSet

Cambiar el nombre de una columna

- Renombrar columnas es útil para estandarizar o traducir encabezados antes de compartir o exportar.

```
df.rename(columns={"Puntaje": "Nota Final"}, inplace=True)  
print(df)
```

Agregar nueva columna

- Puedes **crear columnas nuevas desde listas** (misma longitud que el DataFrame).
- Esto es clave cuando agregas resultados, clasificaciones, o transformaciones.

```
df["Puntaje"] = [90, 85, 88, 92, 87]  
print(df)
```

Actividad DataFrame

Utiliza el siguiente diccionario para crear un DataFrame con información de 6 estudiantes

```
import pandas as pd

estudiantes = {
    "Nombre": ["Sofía", "Mateo", "Valentina", "Andrés", "Laura", "Tomás"],
    "Edad": [22, 24, 21, 23, 22, 25],
    "Programa": ["Ingeniería", "Psicología", "Medicina", "Derecho", "Arquitectura", "Contaduría"]
}

df = pd.DataFrame(estudiantes)
```

1. Muestra las primeras 3 filas del DataFrame.
2. Muestra las últimas 2 filas.
3. Usa **.info()** para ver un resumen estructural.
4. Usa **.describe()** y analiza lo que devuelve. ¿Qué columnas aparecen?
5. Agrega una nueva columna llamada "Nota Final" con estos datos:[4.5, 3.9, 4.2, 3.7, 4.8, 3.5]
6. Cambia el nombre de la columna "Nota Final" a "Promedio".

Seleccionar columnas

Una sola columna

```
df["Nombre"]
```

Devuelve una **Series** con los datos de la columna "Nombre".

Varias columnas

```
df[["Nombre", "Edad"]]
```

Devuelve un nuevo **DataFrame** con solo esas columnas.

Consejo: Usa doble corchete `[ [ ] ]` cuando selecciones más de una columna.

Acceso a elementos en un DataFrame (.iloc .loc)

`.iloc[]` — Acceso por índice numérico

Usa índices numéricos como si accedieras por posición (igual que en una lista).

```
#Selección de las filas 1, 3, 4
print(df.iloc[[1, 3, 4]])
print("-----")

#Selección del registro(fila) 3 columna("Edad") 1
print(df.iloc[3, 1])
print("-----")

#Selección de un rango de filas y columnas
print(df.iloc[0:3, 0:2])
print("-----")

#Selección de todas las filas y la columna con índice 2
print(df.iloc[:, 2])
```

`.loc[]` — Acceso por etiqueta

Usa el índice nombrado del DataFrame (por defecto es numérico, pero se puede personalizar).

```
#Selecciona todas las columnas pero solo el registro 1
print(df.loc[1])
print("-----")

#Selecciona la fila 1, columna "Nombre"
print(df.loc[1, "Nombre"])
print("-----")

#Selecciona un numero de filas especifico y columnas especificas
print(df.loc[0:2, ["Nombre", "Programa"]])
print("-----")

#Filtrar datos por condición
print(df[df["Edad"] > 23])
```

Actividad exploración de datos

- **Selección de columnas y filas**

- Muestra la columna "Nombre".
- Muestra las columnas "Nombre" y "Programa".
- Muestra la tercera fila completa usando `.iloc[]`
- Muestra el valor del promedio del estudiante en la primera fila usando `.loc[]`

- **Filtrado por condiciones**

- Muestra todos los estudiantes con edad mayor a 22
- Muestra todos los estudiantes cuyo promedio sea menor a 4.0
- Muestra los estudiantes del programa "Medicina"
- **Muestra los estudiantes que:**
 - Tienen promedio mayor a 4.0 y estudian "Ingeniería" o "Arquitectura".

```
import pandas as pd

datos = {
    "Nombre": ["Sofía", "Mateo", "Valentina", "Andrés", "Laura", "Tomás"],
    "Edad": [22, 24, 21, 23, 22, 25],
    "Programa": ["Ingeniería", "Psicología", "Medicina", "Derecho", "Arquitectura", "Contaduría"],
    "Promedio": [4.5, 3.9, 4.2, 3.7, 4.8, 3.5]
}

df = pd.DataFrame(datos)
```

A background image showing a white shelf filled with numerous colorful binders (blue, yellow, red, green, black) arranged in rows. The binders have white labels and circular holes at the bottom. The text is overlaid on this image.

Cargar y explorar archivos en pandas

Leer archivo **.csv**

1. CARGAR ARCHIVO A COLAB

1. Cargar el archivo de manera manual a colab (opcional)

```
from google.colab import files  
files.upload()
```

2. Cargar el archivo de manera directa al colab

2. MANIPULAR ARCHIVO .CVS

1. `read_csv()` lee archivos separados por comas.

```
import pandas as pd  
  
df = pd.read_csv("/content/diabetes.csv")  
print(df.head())
```

Leer archivos **.csv** desde una **URL**

```
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv"  
df = pd.read_csv(url)  
print(df.head())
```

Ventajas:

- No necesitas descargar ni subir archivos.
- Puedes trabajar con datasets públicos disponibles en GitHub, Kaggle o instituciones.
- Perfecto para Colab, ya que está conectado a internet.

Leer archivos **.xlsx** (Excel)

```
df = pd.read_excel("/content/Date_Fruit_Datasets.xlsx", sheet_name="Date_Fruit_Datasets")  
print(df.head())
```

- Puedes leer archivos de Excel con múltiples hojas.
- Si no se especifica **sheet_name**, lee la primera por defecto.

Actividades para cada archivo

- Realiza los siguientes pasos para `df_csv`, `df_url` y `df_excel`:
 - Muestra las primeras 5 filas.
 - Muestra las últimas 3 filas.
 - Usa `.info()` para ver un resumen estructural del DataFrame.
 - Usa `.describe()` para obtener estadísticas descriptivas.
 - Muestra desde la fila 10 hasta la 21 y todas las columnas completas usando `.iloc[]`.
 - Muestra el valor de columnas específica en la fila 22 usando `.iloc[]` y `.loc[]`